

CENG113M Week 11

Lab 10 — Functions II

Recursion • Functions Calling Functions • Modularity

PART 1

Recursion

Trace the Recursive Call

```
def countdown(n):  
    if n == 0:  
        print("Go!")  
        return  
    print(n)  
    countdown(n - 1)  
  
countdown(4)
```

What is printed when
countdown(4) is called?

Write the output **in order**, one
value per line.

Line 1: _____

Line 2: _____

Line 3: _____

Line 4: _____

Line 5: _____

Trace the Recursive Call

```
def countdown(n):  
    if n == 0:  
        print("Go!")  
        return  
    print(n)  
    countdown(n - 1)  
  
countdown(4)
```

What is printed when
countdown(4) is called?

Write the output **in order**, one
value per line.

Line 1: 4

Line 2: 3

Line 3: 2

Line 4: 1

Line 5: Go!

Factorial

The factorial of n is defined as:

$$n! = n \times (n - 1) \times \cdots \times 1, \quad 0! = 1$$

Write a **recursive** function `factorial(n)` that computes $n!$. Do **not** use loops.

```
print(factorial(0))    # 1
print(factorial(1))    # 1
print(factorial(5))    #
    120
print(factorial(7))    #
    5040
```

Every recursive function needs:

- 1) a *base case* that stops the recursion
- 2) a *recursive case* that moves toward it.

Fibonacci

The Fibonacci sequence is defined as:

$$F(0) = 0, \quad F(1) = 1, \quad F(n) = F(n-1) + F(n-2) \text{ for } n \geq 2$$

Write a **recursive** function `fibonacci(n)` that returns the n -th Fibonacci number.

```
print(fibonacci(0))    # 0
print(fibonacci(1))    # 1
print(fibonacci(6))    # 8
print(fibonacci(10))   #
55
```

This function has **two** base cases.

What are they?

Recursive Power

Write a **recursive** function `power(base, exp)` that computes base^{exp} .

You may **not** use Python's `**` operator or `math.pow`.

```
print(power(2, 0))      # 1
print(power(2, 10))     #
    1024
print(power(3, 4))      #
    81
print(power(5, 3))      #
    125
```

Think about it:

$$\text{base}^0 = 1$$

$$\text{base}^n = \text{base} \times \text{base}^{n-1}$$

PART 2

Functions Calling Functions

Read the Code

```
def is_even(n):  
    return n % 2 == 0  
  
def keep_evens(numbers):  
    result = []  
    for n in numbers:  
        if is_even(n):  
            result.append(n)  
    return result  
  
print(is_even(7))  
print(keep_evens([1, 2, 3, 4,  
    5, 6]))
```

What does each line print?

Line 1: _____

Line 2: _____

`is_even` is a **helper** — it is called *inside* `keep_evens`, but can also be used anywhere else in the program.

Read the Code

```
def is_even(n):  
    return n % 2 == 0  
  
def keep_evens(numbers):  
    result = []  
    for n in numbers:  
        if is_even(n):  
            result.append(n)  
    return result  
  
print(is_even(7))  
print(keep_evens([1, 2, 3, 4,  
                  5, 6]))
```

What does each line print?

Line 1: **False**

Line 2: **[2, 4, 6]**

`is_even` is a **helper** — it is called *inside* `keep_evens`, but can also be used anywhere else in the program.

BMI Report

Write two functions: a **helper** `calc_bmi(weight_kg, height_cm)` and a **main** function `bmi_report(weight_kg, height_cm)` that calls it.

- ▶ `calc_bmi` computes and **returns** $BMI = w / (h/100)^2$.
- ▶ `bmi_report` calls `calc_bmi`, then prints the BMI (1 decimal) and its category: Underweight (<18.5), Normal ($18.5-24.9$), Overweight ($25-29.9$), Obese (≥ 30).

```
bmi_report(70, 175)      # BMI: 22.9    --> Normal
bmi_report(50, 175)      # BMI: 16.3    --> Underweight
print(calc_bmi(110, 170)) # 38.1      (helper works on its own
                                too!)
```

Grade Report

Write a **helper** function `average(lst)` that returns the mean of a list, and a **main** function `grade_report(scores)` that uses it.

Score	Grade
90–100	AA
80–89	BA
70–79	BB
60–69	CB
0–59	FF

```
grade_report([85, 90, 72, 65])  
# Average: 78.0 --> BB  
  
grade_report([95, 98, 92])  
# Average: 95.0 --> AA  
  
print(average([10, 20, 30])) # 20.0
```

average is useful on its own — that is the point of a good helper function.

PART 3

Modularity

Spot the Design Problem

```
def run():  
    name = input("Name: ")  
    age = int(input("Age: "))  
    if age < 0 or age > 120:  
        print("Invalid age!")  
        return  
    year = 2026 - age  
    print(f"Hi {name}!")  
    print(f"Born in: {year}")  
  
run()
```

This code works, but it is **not modular**.

Identify **three** separate responsibilities packed into `run()`:

1. _____
2. _____
3. _____

Spot the Design Problem

```
def run():  
    name = input("Name: ")  
    age = int(input("Age: "))  
    if age < 0 or age > 120:  
        print("Invalid age!")  
        return  
    year = 2026 - age  
    print(f"Hi {name}!")  
    print(f"Born in: {year}")  
  
run()
```

This code works, but it is **not modular**.

Identify **three** separate responsibilities packed into `run()`:

1. Get user input
2. Validate the age
3. Compute & display result

Refactor into Modules

Take the `run()` function from **Ex 08** and split it into four separate functions:

- > `get_user_info()`
reads and returns name and age
- > `validate_age(age)`
returns True if age is between 0 and 120
- > `calculate_birth_year(age)`
returns 2026 - age
- > `display_result(name, year)`
prints the greeting and birth year
- > `run()`

Shopping Cart

Build a modular shopping cart program. Each responsibility gets its own function:

- ▶ `get_items()` — asks for item name and price until the user enters an empty name; returns a list of (name, price) tuples.
- ▶ `apply_discount(price, rate)` — returns the discounted price.
- ▶ `calculate_total(items)` — sums all prices and adds 18% VAT.
- ▶ `print_receipt(items, total)` — prints a formatted receipt.
- ▶ `main()` — orchestrates everything; contains no calculations.

Test each function **independently** before wiring them together in `main()`.

Putting It All Together

Build a **number guessing game** that combines all three topics.

- ▶ Use **modularity**: separate functions for generating the secret number, getting a valid guess, giving a hint ("Too high" / "Too low"), and displaying the final result.
- ▶ Write a **helper** function `build_hint(guess, secret)` that returns "Too high" or "Too low", and appends " (close!)" if the guess is within 5 of the answer. Call it from your hint-giving function.
- ▶ Use **recursion** for the game loop: after a wrong guess, the function calls itself with the updated attempt count instead of using a loop.